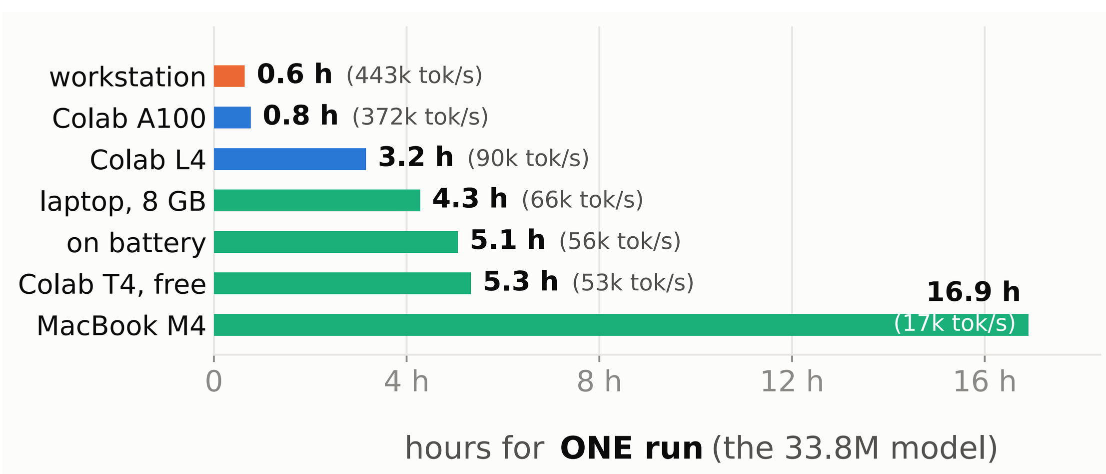
UNIVERSITY *of* WASHINGTON

CSED 505: BUILDING A MODEL FACTORY

501 the statistics · 502 the mechanics · 503 the language stack · 504 scale — All four end when a model finishes training. None covers needing a hundred models and having to believe the differences between them.

Jeffrey Stall · Patrick Kwok · Leon Wan — CSED 504. This board is the machinery; the upper board is the experiment it served.

You do not need the workstation



An **8 GB laptop GPU beats the free tier, even unplugged**. The two paid Colab tiers bill the same — **$111 for 7 days** against **$112 for 30** — so the faster tier buys time, not money. Electricity for all 148 GPU-hours was **$7**; the workstation that ran them, **$24,000**.

When memory is a wall, and when it is only a speed limit

What must fit is the model — about 12–16 bytes per parameter. Our 98M needs **1.6 GB**; a 7B — a small chat assistant — needs **84–112 GB**, beyond any consumer card.

Everything above the floor is batch, and batch is optional. Accumulation runs one big step as smaller passes — **the same update**, within 2%.

So an 8 GB laptop trains the very step a 96 GB card trains. A 10 GB run folds into **6.1 GB at 28,027 tok/s** — the same model.

Nobody on the chart beside this needed 80 GB. 96, 80, 24, 16, 8 GB — all ran the same step. The A100 sells **speed**, not room.

And Windows does not warn you. An oversized batch silently spills into system RAM: **28,027 → 5,075 tok/s**.

The factory, in numbers

197 pretraining runs, every one from scratch

892 fine-tuning runs on top of them, across two tasks

17 languages measured, 12 of them pretrained from scratch

148 GPU-hours, two cards, 71 kWh at the wall

62,500 steps a run — 1.024B tokens of updates, the unit that makes two machines comparable

0 identity collisions, because a vocabulary carries a hash

Budget in tokens, not epochs

Epochs were right in 502–504: the dataset was fixed. Here the dataset *is* the experiment — "40 epochs each" gives one arm **256× the compute** of another.

So fix the total number of tokens, and let epochs fall out. Every run is **1.024B tokens**: the 4M corpus is seen **256 times**, the 1,024M once.

Tokens ÷ tok/s = the wall clock. 1.024B tokens ÷ 443k tok/s ≈ **38 min** — the top bar of the chart above.

A truncated run is a different experiment. The schedule anneals to zero at the *planned* end — a stopped run is caught mid-schedule, not half as good.

Our own records carried the bug. The epoch field reads **125**; the truth is **16 passes**.

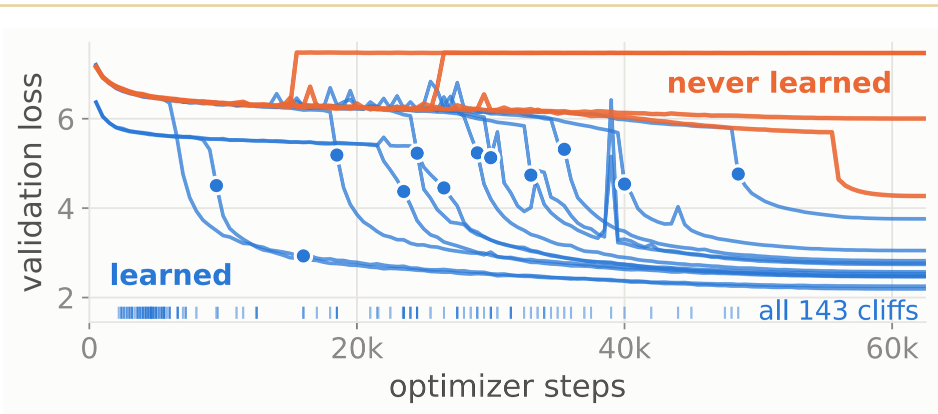
What we made faster

2.07× on the four-cell benchmark — only **1.32×** of it efficiency; the rest is a second card nobody had used. **A third of the twenty were rejected**, and a list of only the wins would be marketing rather than a log. Every row had to make the *same* experiment faster.

WHAT CHANGED	MEASURED	KEPT?
THE DATA PATH		
1 · dataset on the card, no loader	13.8k → **18.5k img/s**	yes
2 · images as uint8, cast per batch	4× less memory, free cast	yes
3 · augment on the GPU	no PIL, no host copy	yes
4 · tokenize once, up front	53 s vs 85 min — **96×**	yes
5 · token dtype from the vocabulary	16k → **2 bytes a token**	yes
PRECISION AND KERNELS		
6 · bf16 autocast, not fp16	**1.34×** on sm_89, no loss-scale	yes
7 · channels_last for CNNs, bf16 only	fp16 + it is **3.5× slower**	guarded
8 · channels_last for ViTs	a no-op on a transformer	no
9 · TF32 for matmul and cuDNN	free on Ampere and later	yes
10 · cudnn.benchmark = True	worth it when shapes are fixed	yes

WHAT CHANGED	MEASURED	KEPT?
11 · fused SGD and AdamW	one kernel, not a chain	yes
12 · torch.compile	no Triton on Windows	can't
13 · attention backend	already sdpa	no change
HOST-SYNC DISCIPLINE		
14 · drop the per-step .item(), CIFAR	**~7%**	yes
15 · the same at MLM scale	**2%** — a live loss is worth it	no
16 · zero_grad(set_to_none=True)	skips a full zero-fill	yes
BATCHING AND SCHEDULING		
17 · batch 64 → 128	**1.31×** — 2048 is *slower*	yes
18 · checkpointing for speed	peak is **3.3 GB of 96**	no
19 · use the second card	**91% / 93%** busy at 300 W	yes
20 · longest-budget-first queue	**2.55×** projected, 2.07 seen	yes

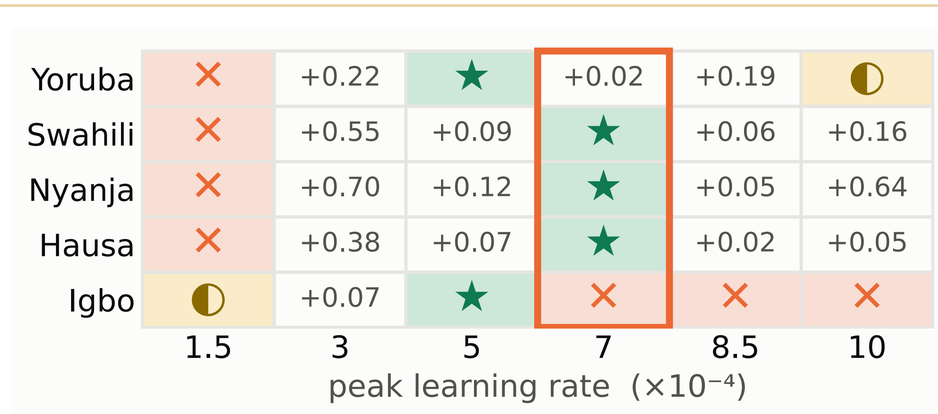
You cannot judge a run early



Yoruba, English and ten more, all on one axis. Every run that learns sits flat and then falls off a cliff — and **no two fall at the same step**: earliest **2,200**, latest **48,500**.

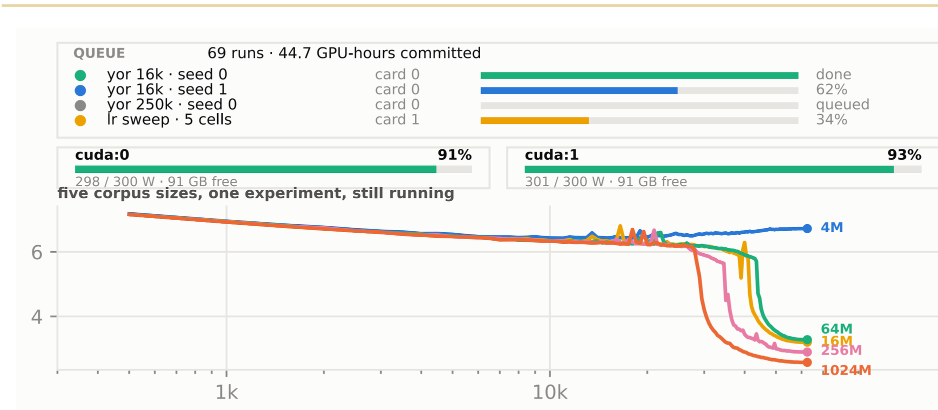
So "still flat at step k" is evidence of nothing. **35 of 197 never learned.**

The setting four languages want destroys the fifth



Five languages, six learning rates, sixty runs, each cell scored against **that language's own best**: ★ its winner, × far behind it, ◐ one seed of two. **Four land at their best in the outlined column. Igbo collapses there** — the same setting, a wasted night.

One screen, both cards, a rush order at 9 p.m.



The chart is five English corpus sizes separating while they run. On 9 August **44.7 GPU-hours sat across 69 runs** and Patrick needed a sweep: a fleet pins to one card, so it took **card 1** while the queue kept card 0. **Twenty minutes** to running, on a measured promise.

A hundred runs is a filing problem first

A study is declared as a **grid**, languages × rates × seeds, never typed run by run; the queue orders it **longest-budget-first**.

Every run writes a record naming **everything that changes the answer**: data fingerprint, vocabulary, settings.

"What have we tried?" is a **query**: **1,089 runs later, no collisions, no archaeology.**

A result has to beat your own noise

Run the same recipe twice and it lands in two different places. That is the seed, not the change you made.

So we measured **our own noise first** and made every claim clear it by **2.27×**. At three runs a side, no test can call anything significant.

The spread is a property of **each cell**, never a constant we reused. **Two of our own claims did not clear it. We retired them.**

Ethics: what we do not claim, and what is next

Nobody on this team reads Yoruba — every judgement here is a benchmark number. And the corpus is scraped web text nobody consented to.

The factory is itself an ethics tool: **sweeping seeds avoids one lucky draw**, each language at its own best.

Our gate prints **9 claims: 6 supported, 2 not, 1 underpowered**. Next: a corpus audit, the evaluation we could not do.

A factory that makes one thing is not a factory

English is the ruler. *Why:* the one language where data was never the constraint. *Learned:* **more data stops helping past 64M tokens** — so Yoruba's 69M-token corpus is not the bottleneck it looks like.

French, Indonesian, Mandarin are the control. *Why:* the borrowed vocabulary knows these three, so they test mere unfamiliarity. *Learned:* they pay **1.04, 1.01, 0.95** tokens where a language's own vocabulary pays one. **Yoruba pays 1.76** — second-highest of seventeen.

Twelve African languages are the gradient. *Why:* one language is an anecdote; seventeen make a slope. *Learned:* the cost tracks **vocabulary coverage**, not geography — **Wolof, uncovered yet cheap at 1.31**, proves it.

Nine functions, one call per language — **twelve pretrained in 48 minutes**.

Sources

A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," ACL, 2020.

M. Marone et al., "mmBERT: A modern multilingual encoder," arXiv:2509.06888, 2025.

Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

D. I. Adelani et al., "SIB-200," EACL, 2024.

D. I. Adelani et al., "MasakhaNER 2.0," EMNLP, 2022.

G. Penedo et al., "FineWeb2: One pipeline to scale them all," arXiv:2506.20920, 2025.

AI statement. We used Claude Code throughout to turn ideas into working code. It made the building faster and the verifying no faster at all — every number on this board is regenerated from committed records by a gate that fails the build when a claim and a record disagree. · Built with PyTorch and HuggingFace tokenizers, plus NumPy and matplotlib; no training framework above them — the loop, the masking, the scheduler and the run records are this project's own. Long jobs ran on a 2 × RTX PRO 6000 Blackwell workstation, benchmarked against Colab A100 / L4 / T4, a MacBook Pro (M4 Pro, MPS) and an 8 GB laptop. · Nineteen references, in full in report 09 §References. · Source: reports/12-the-bottom-board.md · every number on this board regenerates from committed runs/ records